



Multi-Tenant Marketing Intelligence Dashboard

Technical Implementation Plan & Proposal

Prepared for: Client RFP — Multi-Tenant SaaS Dashboard

Version: 1.0

Date: September 2026

Classification: Confidential — Client Proposal

Alphabetum Private Limited

 www.alphabetumtech.com

 [LinkedIn: Alphabetum Technology](#)

Prepared by: Shivani Gupta, CTO

 [LinkedIn: Shivani Gupta](#)

 shivani@alphabetumtech.com

 +91 9897577556



Table of Contents

- | | |
|--------------------------------------|--|
| 1. Executive Summary | 1. Security Architecture |
| 2. Solution Architecture Overview | 2. Scalability & Performance |
| 3. Technology Stack & Justification | 3. Testing Strategy |
| 4. Multi-Tenancy Strategy | 4. CI/CD & Deployment |
| 5. Database Design | 5. Phased Delivery Plan |
| 6. Authentication & Authorization | 6. Risk Assessment & Mitigation |
| 7. Data Pipeline Architecture | 7. Cost Estimation |
| 8. Frontend & Dashboard Architecture | 8. Post-Launch & Maintenance |
| 9. Natural-Language Insights Layer | 9. Phase 2 Roadmap |
| 10. Commercial Proposal | 10. Assumptions & Clarification Points |
| 11. Maintenance & Change Requests | 11. Answers to Proposal Questions |

 Explore the Interactive 3D Architecture — See how the proposed Multi-Tenant Marketing Intelligence Dashboard will work before reading the implementation plan: [View Interactive Demo](#)



1. Executive Summary

We propose to build a unified marketing intelligence platform — a multi-tenant SaaS dashboard that consolidates marketing data from Meta, Instagram, LinkedIn, and Google Analytics 4 into a single, intuitive interface. The platform eliminates the pain of manually collecting and reconciling reports across fragmented marketing tools, giving marketing teams a single source of truth with historical depth and — in Phase 2 — AI-driven actionable insights.

Value Proposition

Value	Description
Unified Visibility	Single pane of glass across Meta, Instagram, LinkedIn, and GA4 — no more tab-switching or CSV exports
Historical Durability	A real stored data layer. Metrics persisted beyond platform API retention limits, enabling true YoY comparisons
Automated Sync	Configurable scheduling (1–24hr) with auto token refresh. Zero manual intervention under normal operation
Enterprise Multi-Tenancy	Database-level tenant isolation (PostgreSQL RLS) from day one — not bolted on later
AI-Ready Architecture	Natural language query engine designed into the data model, deployable after core dashboard stabilizes
Persona-Driven UX	Purpose-built views for Marketing Managers, CMOs, Business Owners, and Agencies with multi-client support

Delivery Approach

We execute across six sequenced phases over 16 weeks, following the client's stated priority order:

Foundation → Integrations → Pipeline → Dashboard → Reliability → AI Insights

(W1-3) (W4-6) (W7-8) (W9-11) (W12-13) (W14-16)

Each phase produces a deployable increment with defined acceptance criteria. Multi-tenancy and security are foundational — they are built first, not hardened later.

Key Architectural Decisions

- PostgreSQL with Row-Level Security for tenant isolation at the database kernel level
- NestJS (Node.js/TypeScript) backend for structured, type-safe API development
- Next.js 14 frontend with SSR for performance and SEO-ready dashboards
- BullMQ + Redis for reliable job scheduling and async data pipeline processing
- AWS (ECS Fargate, RDS, ElastiCache) for managed, scalable infrastructure
- LangChain + GPT-4o for the natural-language insights layer with Text-to-SQL

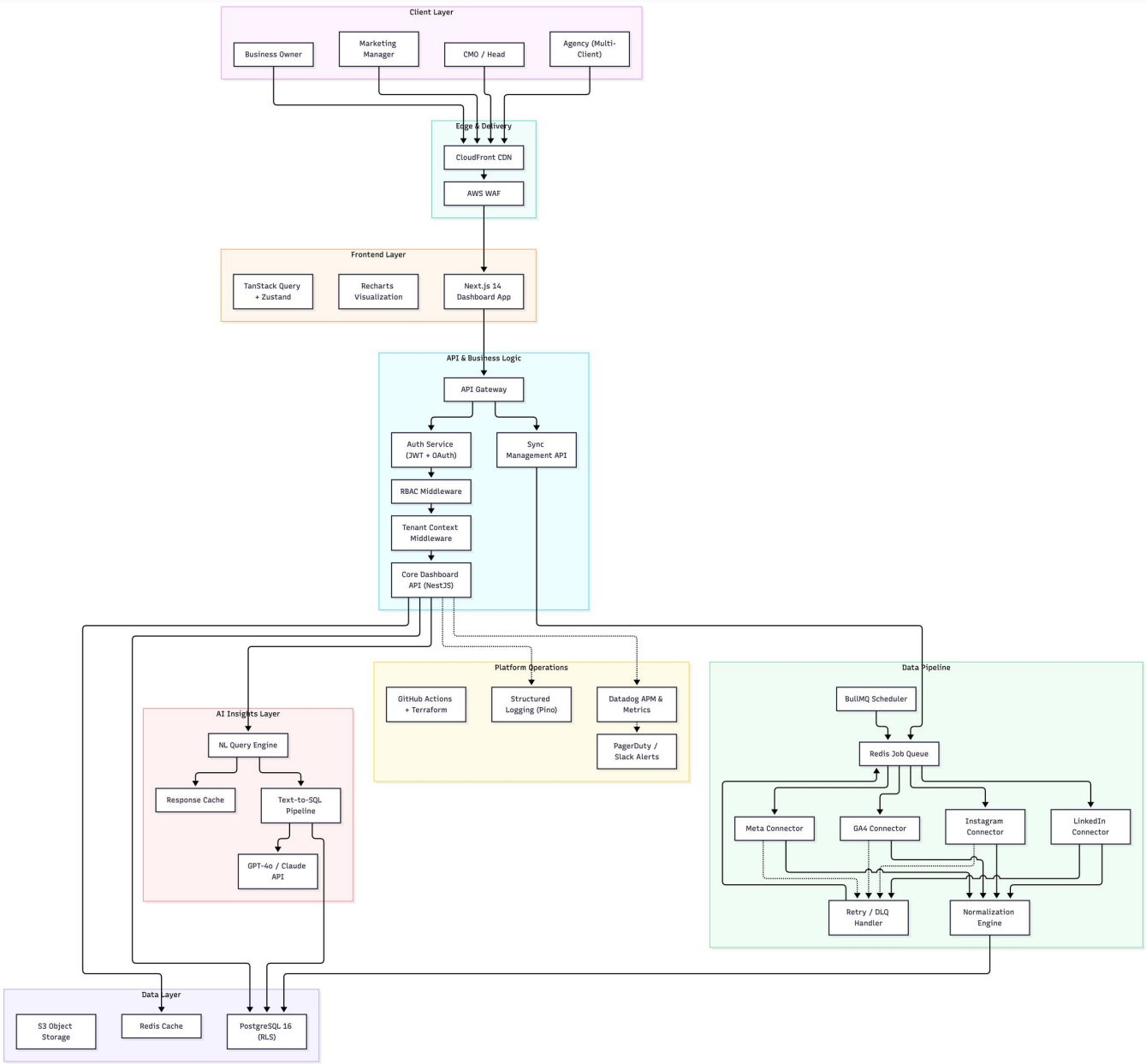


2. Solution Architecture Overview

Design Principles

Principle	Implementation
Data-at-rest, not live-pull	Dashboard reads from stored PostgreSQL data, never calls platform APIs on page load
Tenant isolation as infrastructure	RLS policies enforce isolation at the database level, independent of application logic correctness
Connector abstraction	Each platform integration implements a common interface; adding a new platform is a single module
Horizontal scalability	API servers and pipeline workers scale independently based on load vs. queue depth
Graceful degradation	Token expiry, API outages, and rate limits are handled with retry, backoff, and clear user-facing status

High-Level Architecture



Data Flow Patterns

Pattern 1 — Dashboard Read Path

User → CDN → Next.js SSR → API Gateway → Tenant Middleware → Core API → Redis Cache (hit?) → PostgreSQL → Response

Pattern 2 — Data Ingestion Path

Scheduler → Redis Queue → Worker picks job → Check token → Refresh if needed → Call Platform API → Validate response → Normalize → Upsert to PostgreSQL → Update sync_logs → Emit metric

Pattern 3 — AI Query Path

User query → NL Engine → Inject tenant context + schema → LLM generates SQL → Validate SQL (SELECT only, tenant-scoped) → Execute against read replica → LLM synthesizes natural language response → Cache → Return



3. Technology Stack & Justification

Layer	Technology	Why This Choice for This Project
Backend Framework	NestJS 5 (Node.js 22, TypeScript)	Structured module system with built-in DI, guards, interceptors, and decorators. The module-per-feature pattern maps cleanly to this project's domains (auth, connectors, metrics, sync). First-class TypeScript eliminates an entire class of bugs. Built-in scheduling (@nestjs/schedule) and queue (@nestjs/bullmq) support reduces integration overhead.
Primary Database	PostgreSQL 16	Native Row-Level Security (RLS) — the single most important capability for this project. RLS enforces tenant isolation at the database engine level, meaning even application bugs cannot leak cross-tenant data. JSONB support handles semi-structured platform-specific API responses. Mature partitioning for time-series metrics data.
ORM / Query Builder	Prisma 6	Type-safe query builder with auto-generated TypeScript types from schema. Schema-first migrations align with this project's need for strict, auditable schema evolution. Raw SQL escape hatch for complex analytics queries.
Cache & Queue	Redis 7 (via ElastiCache)	Dual purpose: (1) BullMQ job queue for reliable async pipeline processing with retries, backoff, and dead letter queues, and (2) API response caching to serve dashboard reads from memory when data hasn't changed since last sync.
Frontend Framework	Next.js 14 (App Router, React 18)	SSR for initial dashboard load performance (critical for large datasets). API routes for lightweight BFF pattern. React ecosystem provides the richest charting/visualization library options. Vercel-compatible but self-hostable on AWS.
Charting	Recharts + Tremor	Recharts for custom data visualizations (line, bar, area, pie). Tremor for pre-built dashboard components (KPI cards, sparklines, metric badges). Both are React-native with excellent TypeScript support.
Styling	Tailwind CSS 4 + shadcn/ui	Utility-first CSS for rapid, consistent UI development. shadcn/ui provides accessible, unstyled component primitives that match the "simple, professional" product principle.
Infrastructure	AWS (ECS Fargate, RDS, ElastiCache, S3, CloudFront)	Managed services reduce operational burden. Fargate eliminates server management. RDS provides automated backups, point-in-time recovery, and read replicas. ElastiCache for managed Redis. CloudFront for global edge caching. Well-established pricing model for per-tenant cost estimation.
IaC	Terraform	Declarative infrastructure management. Environment parity (dev/staging/prod) from identical configuration. State locking prevents concurrent drift.
CI/CD	GitHub Actions	Native GitHub integration. Matrix builds for parallel test execution. Environment-based deployment approvals. Reusable workflow definitions.
AI/LLM	LangChain.js + OpenAI GPT-4o	LangChain provides structured prompt templating, chain composition, and output parsing. GPT-4o balances reasoning capability (critical for SQL generation) with cost and latency. Swappable — Claude or Gemini can substitute if cost or capability needs shift.
Observability	Datadog (APM, Logs, Metrics)	Unified platform for traces, metrics, and logs. Pre-built integrations for NestJS, PostgreSQL, Redis. Custom dashboards for sync pipeline health. Alerting with PagerDuty/Slack integration.
Error Tracking	Sentry	Real-time error aggregation with source maps. Release tracking correlates errors to deployments.

Stack Alternatives Considered

Alternative	Why Not
Supabase	Excellent for rapid prototyping but limits control over RLS policy complexity, migration workflows, and background job architecture. The sync pipeline's needs (complex retry logic, rate limit coordination, multi-step normalization) exceed what Supabase Edge Functions offer. Good for simpler CRUD apps, but this project needs fine-grained architectural control.
Firebase	NoSQL data model doesn't suit the highly relational nature of org→account→platform→metric→post hierarchies. Firestore security rules are less expressive than PostgreSQL RLS for complex tenant scoping.
Django/Python	Strong ORM and admin, but the async I/O patterns needed for concurrent platform API calls and real-time dashboard updates are more natural in Node.js. The team's primary expertise is TypeScript-first.
Schema-per-tenant	Operationally expensive: migrations must run against every schema, connection pooling becomes complex, and adding a tenant requires DDL. RLS achieves equivalent isolation with a single schema.



4. Multi-Tenancy Strategy

Approach: Shared Database, Shared Schema, Row-Level Security

We implement database-level tenant isolation using PostgreSQL Row-Level Security (RLS) in a shared schema. Every tenant-scoped table includes an `organization_id` column, and RLS policies enforce that queries can only access rows belonging to the authenticated tenant.

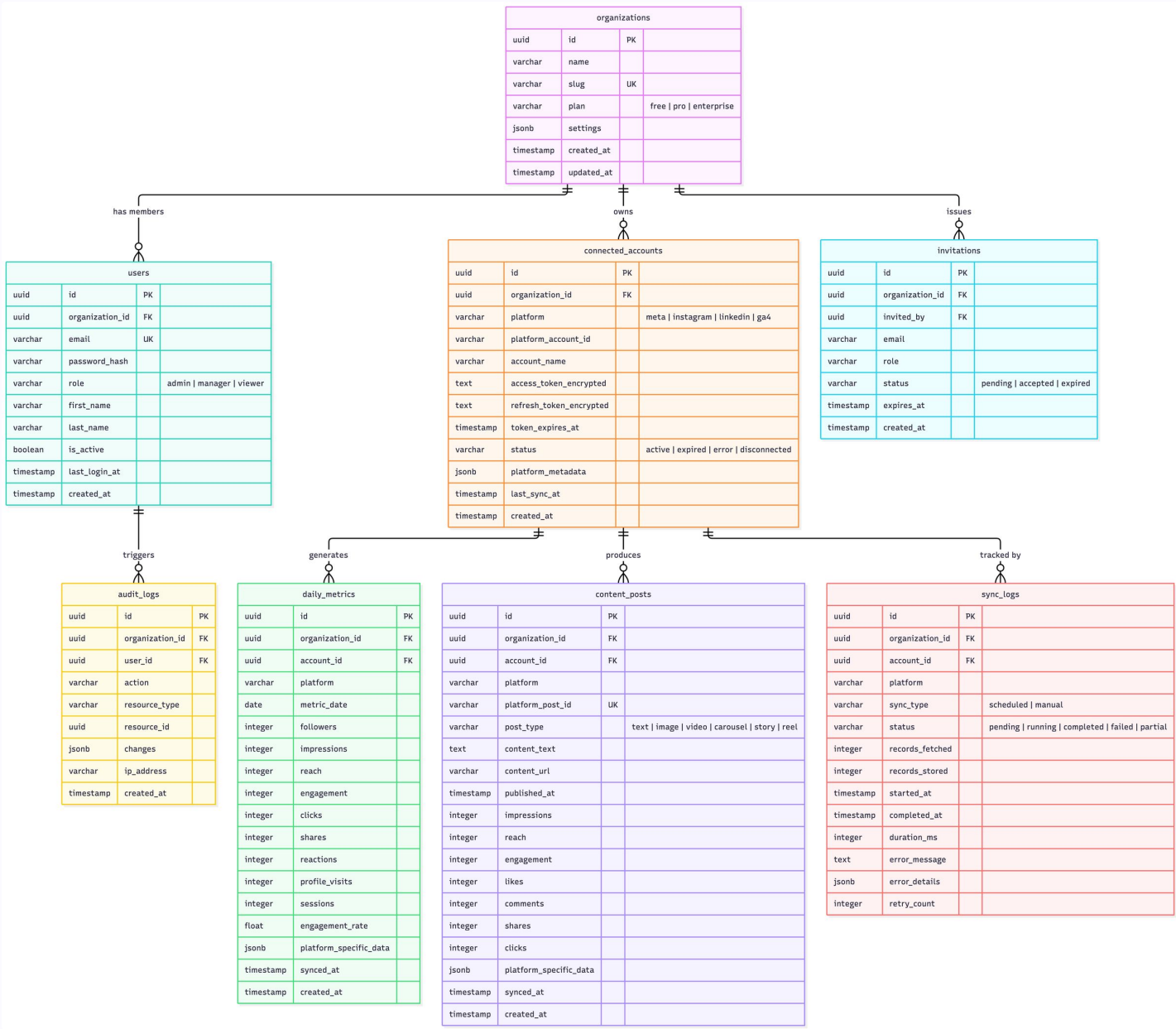
Why RLS Over Alternatives

Approach	Pros / Cons	Verdict
Shared DB + RLS	✔ Single schema to migrate. Database-enforced isolation. No app-layer trust required. Scales to thousands of tenants. Requires careful RLS policy design. Slightly more complex queries.	Selected — best balance of security, operational simplicity, and cost
Schema-per-tenant	Strong isolation. Easy per-tenant backup. Migration nightmare at scale. Connection pool per schema. DDL on tenant creation.	Rejected — operational cost too high beyond ~50 tenants
Database-per-tenant	Maximum isolation. Independent scaling. Extremely expensive. Deployment complexity. Cross-tenant analytics impossible.	Rejected — cost-prohibitive for this use case
App-layer filtering only	Simple to implement. One missing WHERE clause = data leak. No defense in depth.	Rejected — violates the hard isolation requirement



5. Database Design

Entity-Relationship Diagram



Migration Strategy

Tool

Prisma Migrate (schema-first, generates SQL migrations)

Process

Migration files versioned in git. Applied as pre-deploy hook in CI/CD pipeline

Safety

All migrations tested against staging DB with production-like data volume before prod deploy

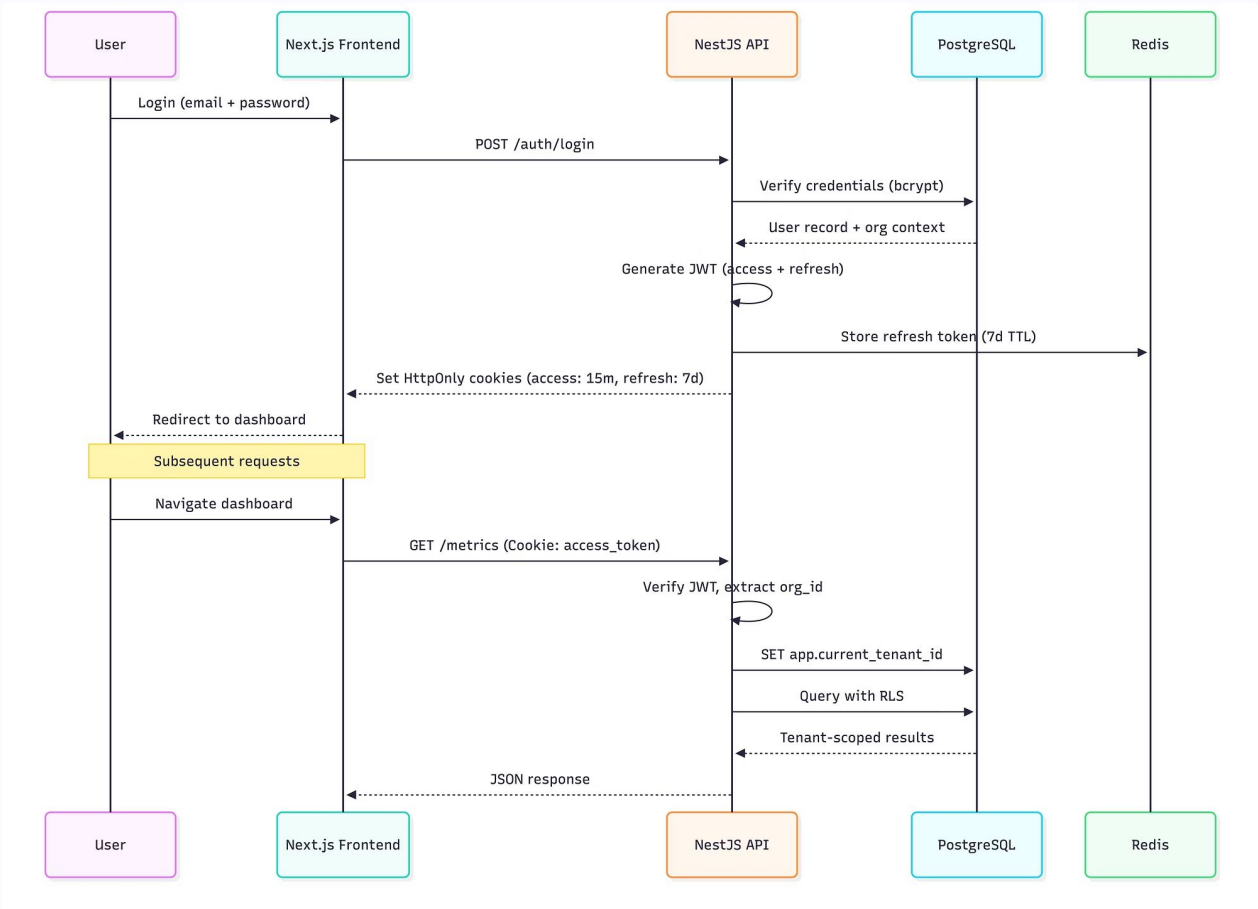
Rollback

Every migration has a corresponding down migration. Destructive changes (column drops) are two-phase: deprecate → verify no usage → remove

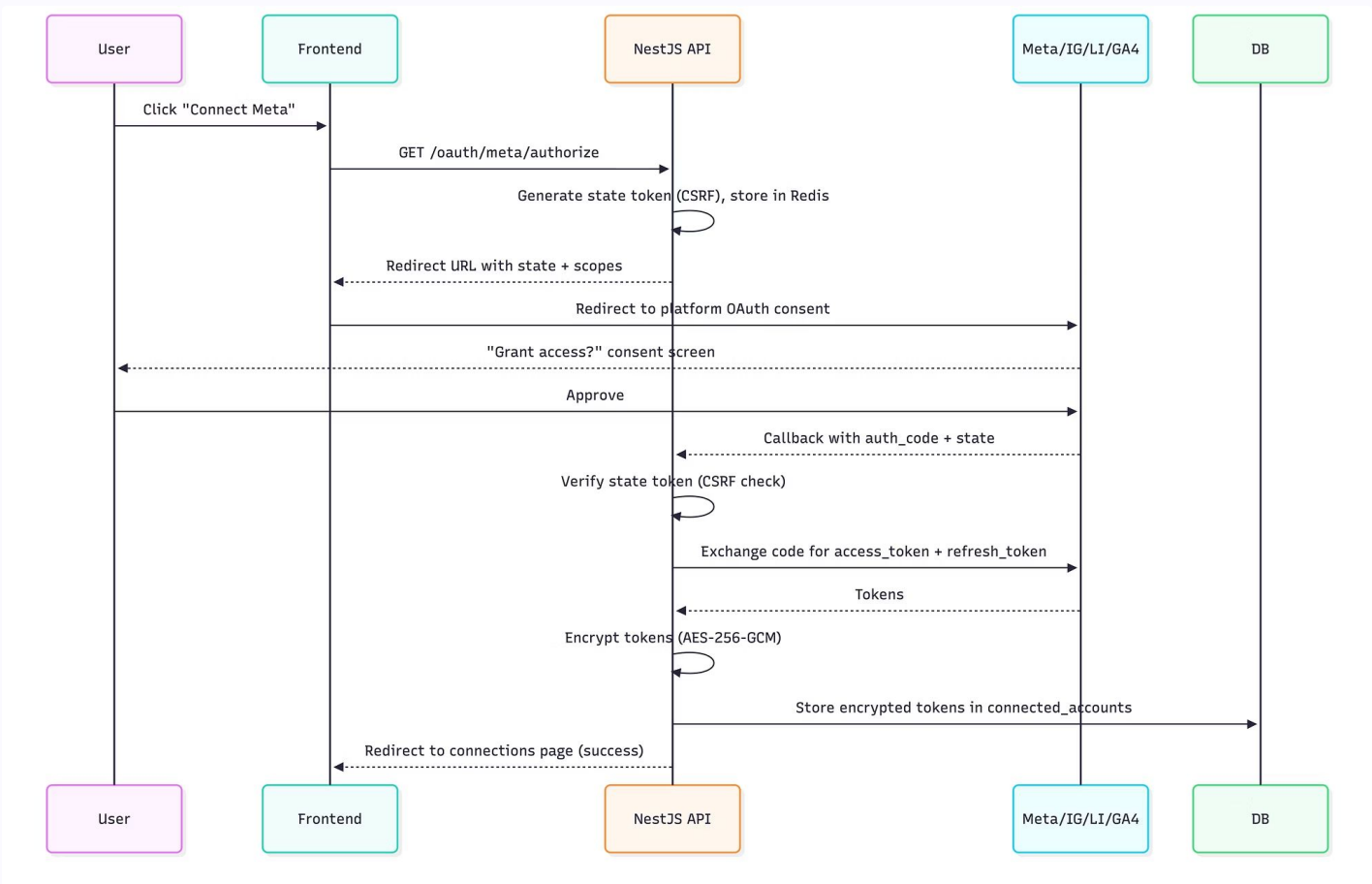


6. Authentication & Authorization

Authentication Architecture



OAuth Flow for Platform Connections



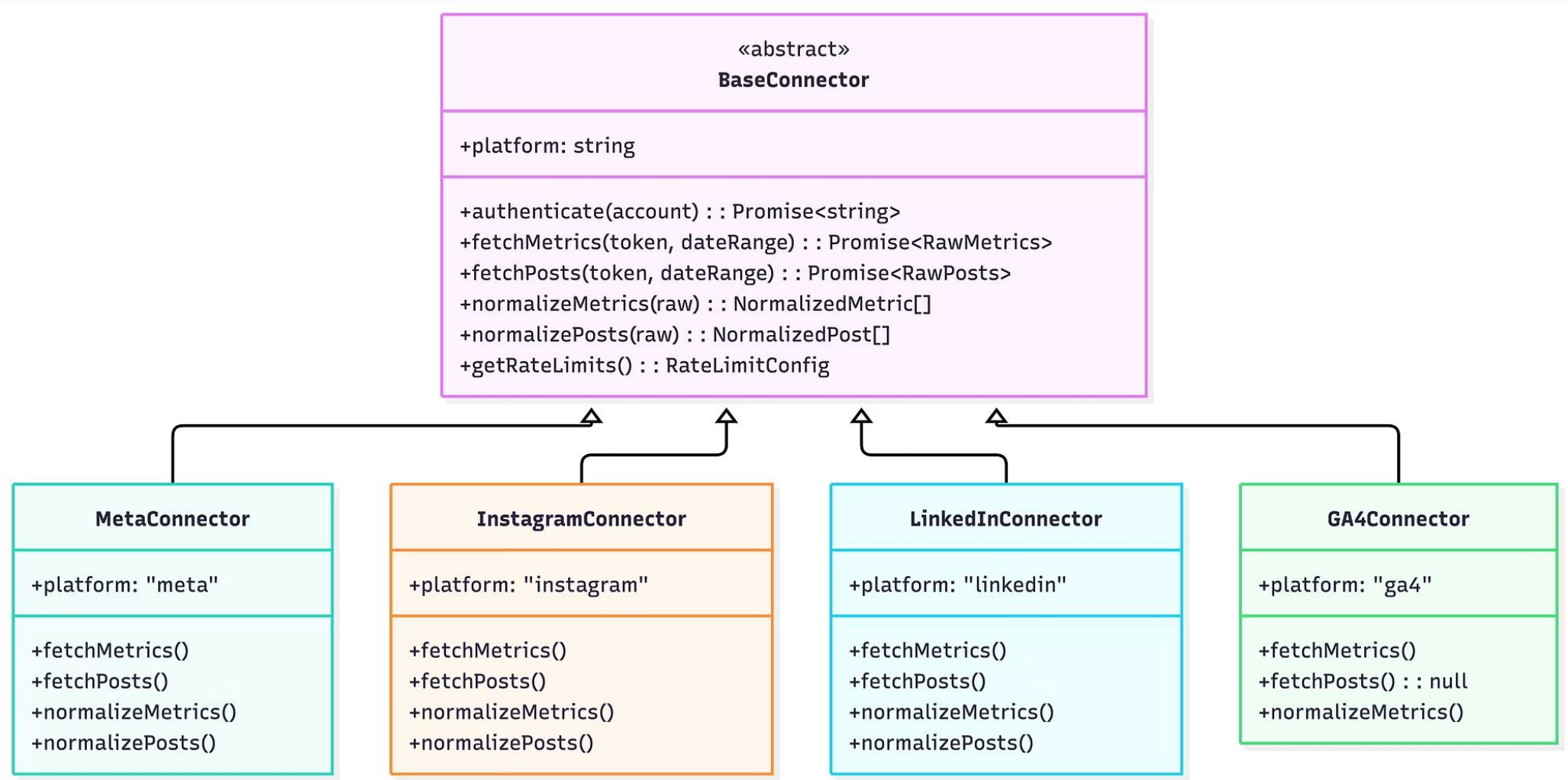
Permission Matrix

Action	Admin	Manager	Viewer
View dashboard & metrics	✓	✓	✓
Filter/export data	✓	✓	✓
Ask AI questions	✓	✓	✓
Trigger manual sync	✓	✓	✗
View sync logs	✓	✓	✗
Connect/disconnect platforms	✓	✗	✗
Invite/remove users	✓	✗	✗
Manage organization settings	✓	✗	✗
Change user roles	✓	✗	✗



7. Data Pipeline Architecture

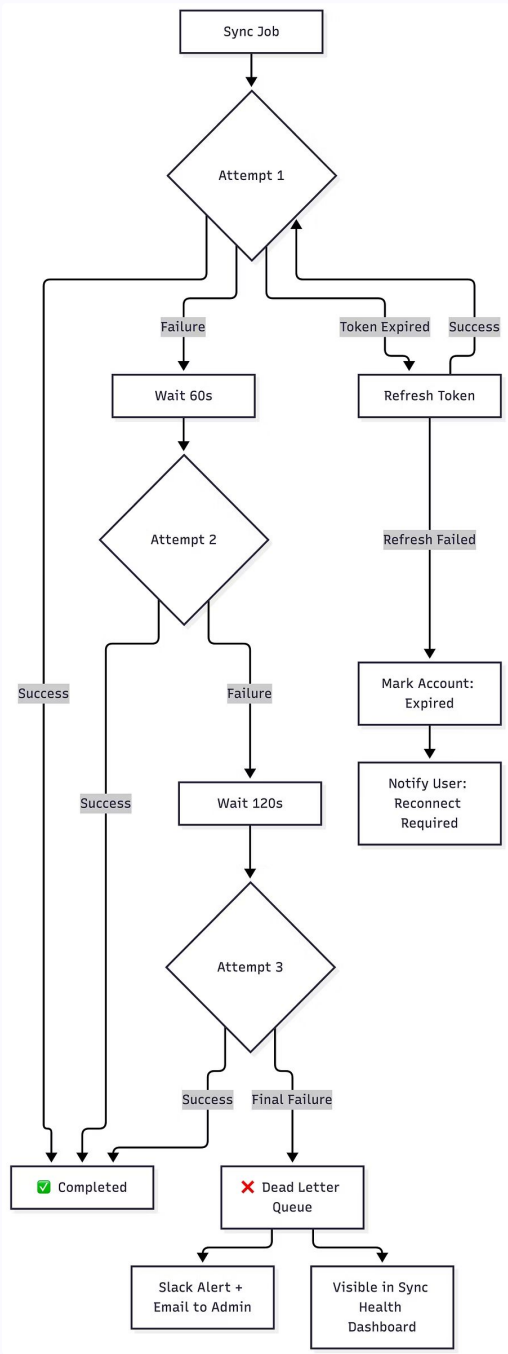
Connector Architecture



Platform API Details

Platform	API Rate Limits	Key Metrics Fetched
Meta	Graph API v19.0 — 200 calls/user/hour	Page followers, post reach, impressions, engagement, post-level likes/comments/shares
Instagram	Instagram Graph API (via Meta) — Shared with Meta quota	Followers, reach, impressions, profile visits, post engagement, story/reel metrics
LinkedIn	Marketing API v2 — 100 calls/day per app	Organization followers, post impressions, clicks, reactions, shares, engagement rate
GA4	Google Analytics Data — 10,000 requests/day per project	Users, sessions, engagement rate, traffic sources, landing pages, device breakdown

Error Handling & Dead Letter Queue



8. Frontend & Dashboard Architecture

Component Architecture

```
src/
├── app/ # Next.js App Router
│   ├── (auth)/ # Auth layout group
│   │   ├── login/
│   │   └── register/
│   ├── (dashboard)/ # Dashboard layout group
│   │   ├── layout.tsx # Sidebar + header + tenant context
│   │   ├── page.tsx # Overview dashboard
│   │   ├── channels/
│   │   │   └── [platform]/ # Platform-specific view
│   │   ├── posts/ # Content performance
│   │   ├── connections/ # Platform connection management
│   │   ├── insights/ # AI natural language query
│   │   ├── sync/ # Sync status & logs
│   │   └── settings/ # Organization settings
│   └── api/ # BFF API routes (optional)
├── components/
│   ├── charts/ # Recharts wrappers
│   │   ├── MetricLineChart.tsx
│   │   ├── ChannelComparisonBar.tsx
│   │   ├── EngagementPieChart.tsx
│   │   └── PostPerformanceTable.tsx
│   ├── dashboard/ # Dashboard-specific components
│   │   ├── KPICard.tsx
│   │   ├── PlatformStatusBadge.tsx
│   │   ├── DateRangePicker.tsx
│   │   └── MetricSparkline.tsx
│   ├── layout/
│   │   ├── Sidebar.tsx
│   │   ├── Header.tsx
│   │   └── BreadcrumbNav.tsx
│   └── ui/ # shadcn/ui primitives
├── hooks/
│   ├── useMetrics.ts # TanStack Query hooks for metrics API
│   ├── useSyncStatus.ts
│   └── useInsights.ts
├── lib/
│   ├── api-client.ts # Axios/fetch wrapper with auth
│   ├── date-utils.ts
│   └── format-metrics.ts
├── stores/
└── ui-store.ts # Zustand for local UI state
```

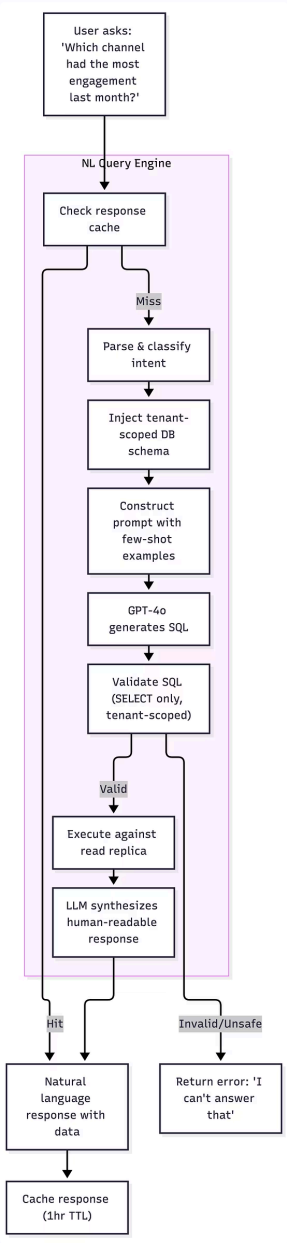
Dashboard Views by Persona

Persona	Default View	Key Metrics	Special Features
Marketing Manager	Channel detail with post-level breakdown	Impressions, engagement, reach by post	Post performance ranking, best time to post analysis
CMO / Marketing Head	Executive overview with cross-channel comparison	Total reach, engagement rate, follower growth, ROI trends	Period-over-period comparison, PDF export
Business Owner	Simplified summary with KPI cards	Total leads, traffic, top engagement	Traffic light indicators (up/down/flat), sparklines
Agency	Multi-client switcher → per-client dashboard	All above, per client	Client selector dropdown, client-branded reports



9. Natural-Language Insights Layer

Architecture



Cost Management

Model	Cost per 1K tokens (input/output)	Estimated monthly cost (100 tenants, 5 queries/day avg)
GPT-4o	\$2.50 / \$10.00	~\$150–250/month
GPT-4o-mini	\$0.15 / \$0.60	~\$15–30/month
Claude 3.5 Sonnet	\$3.00 / \$15.00	~\$200–350/month

Strategy: Use GPT-4o for SQL generation (accuracy matters), GPT-4o-mini for response synthesis (less complex). Cache aggressively. Set per-tenant daily query limits (free: 10, pro: 100, enterprise: unlimited).

Hallucination Mitigation

Schema-constrained prompting

LLM only sees the exact table/column definitions, not the full database

SQL validation

Generated SQL is parsed and validated before execution

Result grounding

The synthesis prompt includes the raw query results — the LLM explains actual data, not hallucinated numbers

Confidence scoring

If the query returns zero rows or the question doesn't map to available columns, the system says "I don't have enough data to answer that" rather than guessing

Show your work

Users can expand to see the generated SQL and raw data table alongside the natural language answer



10. Commercial Proposal

The following is a planning estimate based on the supplied brief. It is intentionally presented as a phase-based range until API access, final UX, reporting depth and non-functional requirements are confirmed.

Workstream	Estimated fee (INR)
Discovery & architecture	₹2.0–2.5 L
Multi-tenant foundation + auth + OAuth framework	₹5.0–6.5 L
Four platform integrations	₹8.0–10.0 L
Data pipeline + scheduler + normalization + reliability foundation	₹5.5–7.0 L
Dashboard UI + reporting APIs	₹5.0–6.5 L
Production hardening, QA, security/performance/UAT	₹2.5–3.5 L
Natural-language insights layer	₹7.0–12.0 L
Estimated total	₹40.0–65.0 L

Commercial model recommendation: fixed-price milestones for the agreed Phase 1 scope, with a separate scoped work package for AI insights. Any provider/API scope expansion, new platform, materially new dashboard module or changed acceptance criteria is handled through a change request.



11. Maintenance & Change Requests

Essential

Monitoring, bug fixes, connector health, security patches and limited monthly support

Standard

Essential + higher support coverage, minor enhancements and connector maintenance

Growth

Standard + prioritized enhancements, new reporting capabilities and agreed engineering capacity

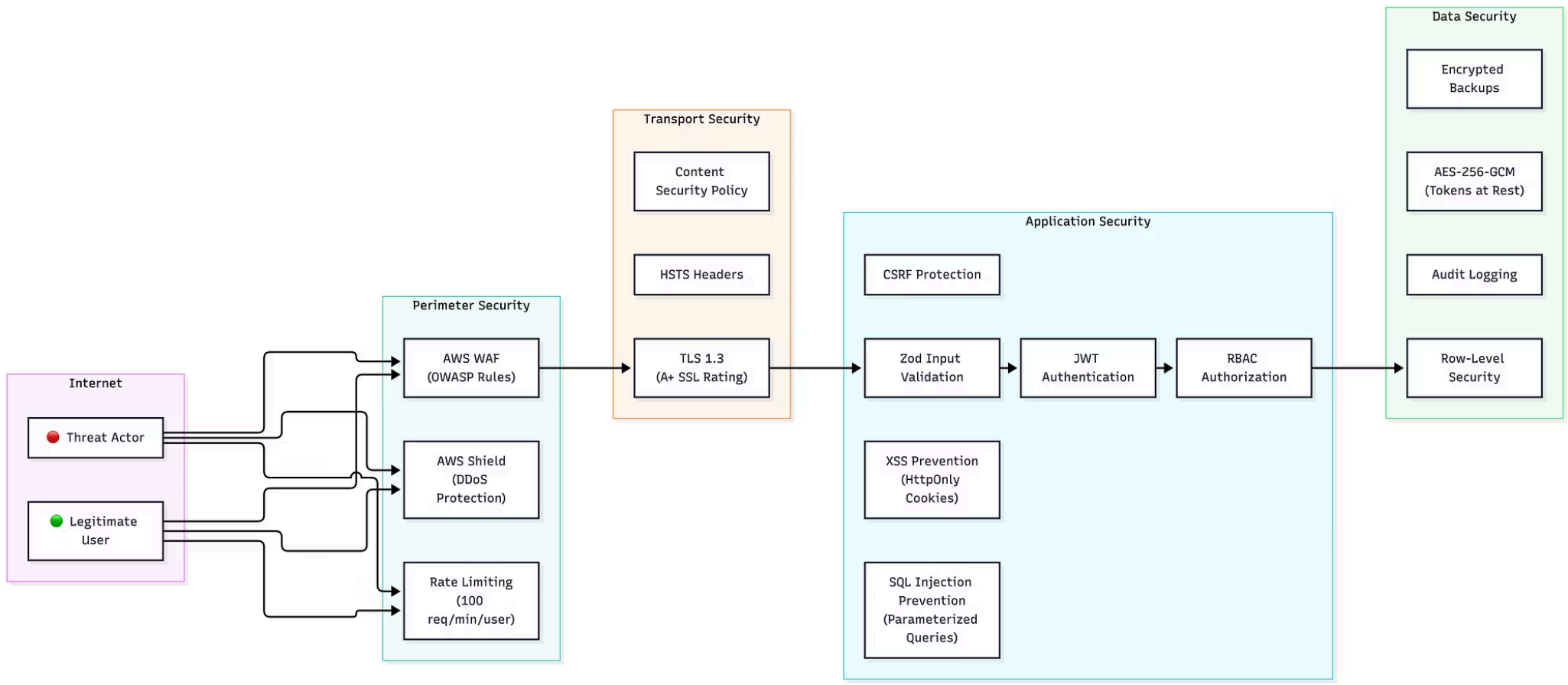
Final maintenance pricing can be set after production traffic and support requirements are known. As a working commercial model, maintenance can be structured as a monthly retainer or as a percentage of implementation value, with third-party API/cloud costs billed separately.

Adding a field or metric: estimate connector impact, normalization changes, storage, API changes, UI and regression scope; then quote as a small change request. Adding a platform: separate connector discovery, OAuth review, API mapping, rate-limit behavior, normalization, QA and production rollout; quote by platform rather than treating it as a minor field change.



12. Security Architecture

Security Layers



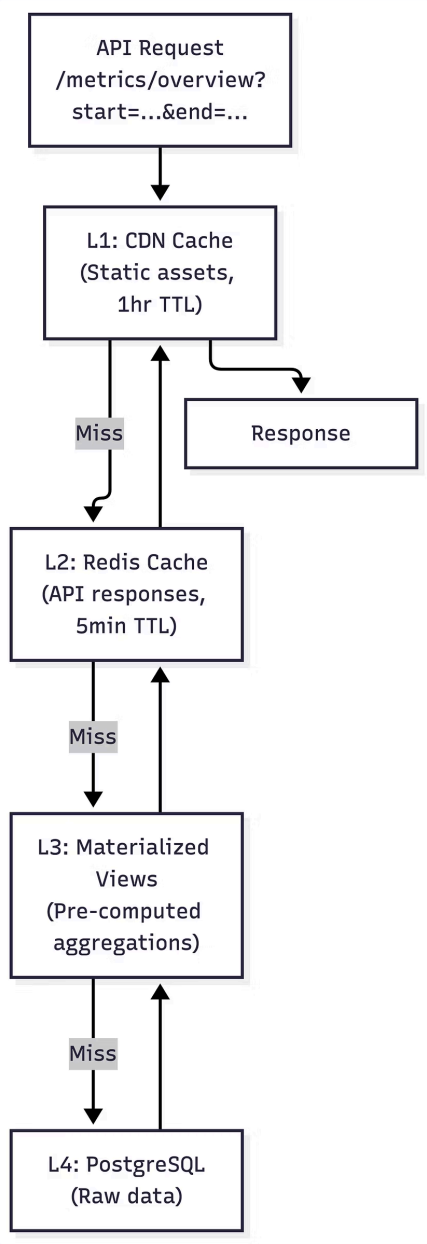
OWASP Top 10 Mitigations

OWASP Risk	Mitigation
A01: Broken Access Control	RLS + RBAC + tenant context middleware + automated isolation tests
A02: Cryptographic Failures	AES-256-GCM for tokens, TLS 1.3 in transit, bcrypt for passwords
A03: Injection	Prisma parameterized queries, Zod input validation, AI SQL validation
A04: Insecure Design	Threat modeling during design phase, principle of least privilege
A05: Security Misconfiguration	Terraform-managed infra, security headers (HSTS, CSP, X-Frame-Options), no default credentials
A06: Vulnerable Components	Dependabot/Snyk automated dependency scanning, weekly updates
A07: Auth Failures	Short-lived JWTs (15m), refresh token rotation, rate-limited login, account lockout
A08: Data Integrity Failures	Signed JWTs, verified OAuth state tokens, pipeline data validation
A09: Logging Failures	Structured audit logging for all admin actions, log integrity monitoring
A10: SSRF	URL allowlisting for platform API calls, no user-controlled URL fetching



13. Scalability & Performance

Caching Strategy



Database Performance

Technique	Implementation
Connection Pooling	PgBouncer (transaction mode, max 100 connections per pool)
Read Replicas	AI query engine and heavy analytics queries routed to read replica
Materialized Views	Pre-computed monthly/weekly rollups refreshed after sync completion
Table Partitioning	daily_metrics partitioned by month; old partitions detached and archived
Query Optimization	Covering indexes on (org_id, date, platform); EXPLAIN ANALYZE on all key queries

Horizontal Scaling

Component	Scaling Trigger	Strategy
API Servers (ECS)	CPU > 70% or P95 latency > 500ms	Auto-scale 2→8 tasks
Pipeline Workers	Queue depth > 100 jobs	Auto-scale 1→4 tasks
PostgreSQL	Read IOPS > 80% capacity	Add read replicas
Redis	Memory > 80%	Upgrade instance size or add cluster nodes

Performance SLAs

Metric	Target	Measurement
Dashboard page load (P50)	< 1.5 seconds	Datadog RUM
Dashboard page load (P95)	< 3.0 seconds	Datadog RUM
API response time (P50)	< 200ms	Datadog APM
API response time (P95)	< 500ms	Datadog APM
AI query response (P50)	< 5 seconds	Custom metric
Data sync latency	< configured frequency + 30min	Sync log analysis
Availability	99.9% monthly	Uptime monitoring



14. Testing Strategy

Test Pyramid

Level	Tool	Coverage Target	What We Test
Unit	Jest	> 80% line coverage	Normalization logic, date calculations, permission checks, encryption/decryption, data transformations
Integration	Jest + Testcontainers	Key flows	API endpoints with real PostgreSQL (RLS verification), Redis interactions, OAuth flows (mocked platforms)
E2E	Playwright	Critical paths	Login → Connect platform → View dashboard → Filter data → Trigger sync → Ask AI question
API Contract	Jest + OpenAPI	100% of endpoints	Request/response shape matches OpenAPI spec
Security	Custom + OWASP ZAP	All endpoints	Tenant isolation, RBAC enforcement, injection attempts, auth bypass attempts
Performance	k6	Key endpoints	Dashboard load < 3s P95, API < 500ms P95, 100 concurrent users

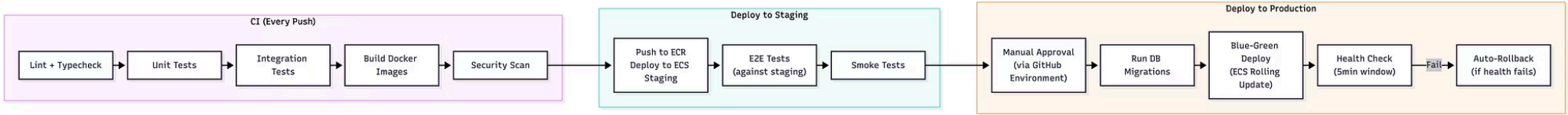


15. CI/CD & Deployment

Repository Structure

```
marketing-intelligence-platform/
├── apps/
│   ├── api/ # NestJS backend
│   │   ├── src/
│   │   ├── prisma/ # Schema + migrations
│   │   ├── Dockerfile
│   │   └── package.json
│   └── web/ # Next.js frontend
│       ├── src/
│       ├── Dockerfile
│       └── package.json
├── packages/
│   ├── shared/ # Shared types, utils, constants
│   └── connectors/ # Platform connector library
├── infra/
│   ├── terraform/ # IaC definitions
│   │   ├── modules/
│   │   ├── environments/
│   │   │   ├── dev/
│   │   │   ├── staging/
│   │   │   └── production/
│   │   └── main.tf
│   └── docker-compose.yml # Local development
├── .github/
│   ├── workflows/
│   ├── ci.yml
│   ├── deploy-staging.yml
│   └── deploy-production.yml
├── turbo.json # Turborepo config
└── package.json # Root workspace
```

Deployment Pipeline



Environment Management

Environment	Purpose	Infrastructure / Data
Local	Developer workstation	Docker Compose (PG + Redis + MinIO); Seed data
Dev	Feature testing	Shared AWS (small instances); Synthetic data
Staging	Pre-production validation	AWS (mirrors prod topology, smaller instances); Anonymized prod snapshot
Production	Live system	AWS (full-scale, multi-AZ); Real data



16. Phased Delivery Plan

Overall delivery window: approximately 16–21 calendar weeks. The 16-week schedule is the base delivery plan; the 17–21-week range allows for provider approvals, integration variability, historical data quality and backfill work, UX refinement, reliability hardening, acceptance testing, and other dependencies. We do not recommend compressing or removing the reliability/data-quality phase because the stored historical layer is central to the product's value and the AI layer depends on trustworthy normalized data.

Overview Timeline — Total: approximately 16–21 calendar weeks

Phase 1: Foundation — 3 weeks (Weeks 1–3)

Goal: Establish the multi-tenant core, authentication, and organizational management

Deliverable	Description	Acceptance Criteria
Project scaffold	Turborepo monorepo with NestJS API + Next.js frontend, Docker Compose for local dev, Terraform for AWS	docker-compose up boots full local stack
Database schema	All core tables created with RLS policies enabled	RLS isolation tests pass (Tenant A cannot see Tenant B)
Auth system	JWT login/logout, refresh token rotation, HttpOnly cookies	Login flow works, tokens rotate correctly
RBAC	Admin/Manager/Viewer role enforcement on all endpoints	Viewer cannot access admin endpoints (403)
Tenant middleware	Automatic tenant context setting on every authenticated request	All queries scoped to correct org
Org management	Create org, update settings, view org details	API endpoints functional with tests
User management	Invite user, list users, change role, remove user	Email invitations sent, role changes applied

Team: 1 Senior Backend + 1 Backend + 1 DevOps

Risk: RLS policy design errors → Mitigated by isolation test suite run on every PR

Phase 2: Platform Integrations — 3–4 weeks (base schedule Weeks 4–6; may extend into Week 7 if provider access or approvals require it)

Goal: Connect to all four platforms with robust OAuth and token management

Deliverable	Description	Acceptance Criteria
OAuth framework	Reusable OAuth flow handler supporting any platform	Connect → consent → callback → tokens stored encrypted
Meta connector	Fetch page metrics + post performance via Graph API	30 days of historical data fetched and normalized
Instagram connector	Fetch account metrics + post performance via IG Graph API	Same as Meta
LinkedIn connector	Fetch org metrics + post performance via Marketing API	Same, accounting for LinkedIn's stricter rate limits
GA4 connector	Fetch analytics data via Data API	Sessions, users, traffic sources, landing pages fetched
Token management	Auto-refresh, expiry detection, reconnect flow	Expired tokens trigger user notification, not silent failure

Team: 2 Backend + 1 Senior Backend (oversight)

Dependencies: Phase 1 auth and tenant system complete

Risk: Platform API rate limits and approval delays → Each connector designed with configurable backoff; developer app registrations started in Week 1

Phase 3: Data Pipeline — 2–3 weeks (base schedule Weeks 7–8; may extend into Week 9 for normalization, backfill, validation, or retry testing)

Goal: Reliable, automated data ingestion with error handling

Deliverable	Description	Acceptance Criteria
Scheduler	BullMQ-based configurable scheduler (1–24hr, default 6)	Jobs dispatched on schedule, configurable per account
Normalization	Platform-specific data mapped to unified daily_metrics schema	All four platforms normalize to identical format
Retry logic	Exponential backoff, DLQ for persistent failures	Failed jobs retry 3x, then land in DLQ with alert
Sync logging	Complete sync history with status, duration, record counts	Sync logs queryable by account, date, status
Manual sync	"Sync Now" button triggers immediate sync	Admin/Manager can trigger, Viewer cannot

Team: 1 Senior Backend + 1 Backend

Dependencies: Phase 2 connectors

Risk: Data quality issues from inconsistent platform APIs → Validation layer with data quality assertions

Phase 4: Dashboard UI — 3–4 weeks (base schedule Weeks 9–11; may extend through Week 12 for UX refinement and acceptance feedback)

Goal: Beautiful, intuitive dashboard serving all four user personas

Deliverable	Description	Acceptance Criteria
Design system	shadcn/ui + Tailwind setup, component library, dark/light mode	Consistent visual language across all pages
Overview dashboard	KPI cards, cross-platform summary, trend sparklines	Shows key metrics at a glance, date range filtering works
Channel views	Per-platform deep-dive with metric charts	Line charts, bar charts, engagement breakdowns per platform
Post performance	Sortable/filterable post table with engagement metrics	Sort by engagement, filter by platform/date/type
Connection management	Connect/disconnect platforms, view connection health	OAuth flows launch from UI, connection status visible
Settings	Org settings, user management, sync frequency configuration	All settings editable by Admin role
Agency multi-client	Client switcher for agency accounts	Agency users can switch between client orgs

Team: 2 Frontend + 1 Designer + 1 Backend (API support)

Dependencies: Phase 3 pipeline populating data

Risk: UI complexity creep → Strict scope control via design mockup sign-off before development

Phase 5: Reliability & Hardening — 2–3 weeks (base schedule Weeks 12–13; may extend through Week 14 for performance, security, recovery, and production-readiness work)

Goal: Production-ready monitoring, performance, and security

Deliverable	Description	Acceptance Criteria
Monitoring	Datadog APM, metrics, logs for all services	Dashboards for API health, sync pipeline, and database
Alerting	PagerDuty/Slack alerts for critical issues	Alerts fire on error spikes, sync failures, resource exhaustion
Load testing	k6 load tests simulating 100 concurrent users	P95 API < 500ms, dashboard load < 3s
Performance optimization	Query optimization, caching, materialized views	All slow queries identified and optimized
Security hardening	OWASP ZAP scan, dependency audit, security headers	No high/critical vulnerabilities, A+ SSL rating
Backup & recovery	Automated RDS backups, tested restore procedure	Point-in-time restore tested, RTO < 1 hour

Team: 1 Senior Backend + 1 DevOps + 1 QA

Dependencies: All functional features complete

Risk: Performance issues under load → Early query profiling in Phase 3–4 reduces late-stage surprises

Phase 6: AI Insights Layer — 3–4 weeks (base schedule Weeks 14–16; may run through Week 21 when earlier phases consume contingency or when AI validation and acceptance require additional time)

Goal: Natural language query engine grounded in real marketing data

Deliverable	Description	Acceptance Criteria
LangChain setup	GPT-4o integration with Text-to-SQL chain	LLM generates valid SQL for test questions
Prompt engineering	Schema-aware prompts, few-shot examples, error handling	90%+ accuracy on benchmark question set
SQL validation	Safety checks on generated SQL (SELECT only, no cross-tenant)	All injection/cross-tenant attack prompts blocked
Chat UI	Natural language input, formatted responses, data tables	Users can type questions and get clear answers
Suggested questions	Context-aware question suggestions based on available data	"Try asking..." prompts appear based on connected platforms
Cost controls	Per-tenant query limits, response caching, model tiering	Costs within \$200–300/month at 100 tenants

Team: 1 Senior Backend (AI/ML experience) + 1 Frontend + 1 Backend

Dependencies: Phase 4 dashboard as hosting surface, Phase 5 monitoring for cost tracking

Risk: LLM accuracy on complex SQL → Extensive prompt testing, fallback to "I can't answer that" over hallucination



17. Risk Assessment & Mitigation

#	Risk	Impact	Probability	Mitigation	Contingency
1	Platform API changes or deprecation	High	Medium	Abstract connector pattern; monitor API changelogs; version-pin API endpoints	Swap connector implementation without affecting rest of system
2	RLS data leak	Critical	Low	Automated isolation test suite on every PR; database-level enforcement	Immediate hotfix capability; audit log review for exposure assessment
3	OAuth token expiry causes silent data gaps	High	Medium	Proactive token refresh; expiry monitoring; clear user-facing "Reconnect" alerts	Admin notifications + sync health dashboard shows gaps
4	Platform API rate limiting throttles sync	Medium	High	Per-platform rate limit tracking; configurable sync frequency; adaptive backoff	Priority queue (manual syncs first); stagger sync times across tenants
5	LLM generates incorrect/misleading insights	High	Medium	SQL validation layer; ground responses in raw data; confidence scoring	Show raw data table alongside AI answer; "I don't have enough data" fallback
6	Scope creep in dashboard UI	Medium	High	Design mockup sign-off before development; strict phase boundaries	Feature backlog for post-launch iteration
7	Platform developer app approval delays	High	Medium	Start platform app registrations in Week 1; use sandbox/test accounts in parallel	Develop against mock APIs; swap in real credentials when approved
8	Database performance under scale	Medium	Medium	Table partitioning; materialized views; connection pooling; read replicas	Vertical RDS scaling as immediate relief; query optimization sprint
9	LLM API costs exceed projections	Medium	Medium	Response caching; model tiering (GPT-4o for SQL, mini for synthesis); per-tenant limits	Switch to cheaper model; reduce cache TTL; increase query limits
10	Key team member unavailability	Medium	Low	Documentation -first development; pair programming on critical paths	Cross-trained team members can cover any component
11	LinkedIn API access restrictions	Medium	Medium	LinkedIn Marketing API requires partner program approval for some endpoints	Start application early; implement with available endpoints first
12	Multi-tenant performance interference	Medium	Low	Connection pooling per tenant; query timeout limits; resource monitoring	Tenant-level rate limiting; isolate heavy tenants to dedicated resources



18. Cost Estimation

Infrastructure Cost (Monthly, at 100 Tenants)

Service	Instance/Tier	Monthly Cost	Notes
AWS ECS Fargate (API)	2 tasks × 0.5 vCPU / 1GB	~\$60	Auto-scales to 8 tasks under load
AWS ECS Fargate (Workers)	1 task × 0.5 vCPU / 1GB	~\$30	Scales with queue depth
AWS RDS PostgreSQL	db.t4g.medium (Multi-AZ)	~\$140	100GB gp3 storage included
AWS ElastiCache Redis	cache.t4g.small	~\$50	Single node (HA in prod)
AWS S3 Standard	—	~\$5	Report storage, backups
AWS CloudFront Standard	—	~\$10	CDN for frontend assets
AWS ALB	Application Load Balancer	~\$25	Plus per-request charges
Datadog Pro plan	5 hosts	~\$115	APM + Logs + Metrics
Sentry Team plan	—	~\$26	Error tracking
OpenAI API	GPT-4o + GPT-4o-mini	~\$150–250	Variable with query volume
Domain + SSL	Route 53 + ACM	~\$5	Certificate free via ACM
Total	—	~\$616–716/month	Per-tenant cost ~\$6–7/month

Scaling Projections

Tenants	Est. Monthly Infra Cost	Per-Tenant Cost	Notes
10	~\$400	~\$40	Baseline infrastructure floor
100	~\$650	~\$6.50	Optimal efficiency range
500	~\$1,200	~\$2.40	RDS upgrade, additional workers
1,000	~\$2,500	~\$2.50	Read replica, larger cache, more Fargate tasks
5,000	~\$8,000	~\$1.60	Partitioning, dedicated resources for hot tenants

❏ NOTE AI costs scale linearly with query volume, not tenant count. A tenant making 100 AI queries/month costs more in LLM tokens than one making 5. Per-tenant query limits and aggressive caching are critical cost controls.



19. Post-Launch & Maintenance

SLA Commitments

Metric	SLA	Measurement
Platform uptime	99.9% monthly (< 44min downtime)	Synthetic monitoring
Data sync freshness	Within configured frequency + 30min	Sync log analysis
API response time (P95)	< 500ms	Datadog APM
Critical bug response	< 4 hours (business hours)	Ticketing system
Security vulnerability patch	< 24 hours for critical CVEs	Dependency scanning

Backup & Disaster Recovery

Component	Backup Strategy	RPO	RTO
PostgreSQL	Automated RDS snapshots (daily) + continuous WAL archiving	5 minutes (PITR)	< 1 hour
Redis	RDB snapshots (hourly)	1 hour	< 15 minutes
S3	Cross-region replication	Near-zero	< 30 minutes
Application code	Git (GitHub)	Zero	< 30 minutes (rebuild + deploy)
Terraform state	S3 backend with versioning + state locking	Zero	< 15 minutes

Change Request Process

01	02
Request	Triage
Client submits via dedicated channel (email/portal)	Assess scope — is it a bug fix, minor change, or feature?
03	04
Scope & estimate	Approval
For changes beyond maintenance (e.g., adding a new platform), provide effort estimate within 48 hours	Client approves estimate
05	06
Implementation	Billing
Follows standard CI/CD pipeline (develop → test → staging → production)	Bug fixes within SLA scope are included. Feature additions billed at agreed hourly rate or mini-SOW

Platform API Monitoring

Each social platform periodically changes or deprecates API endpoints. We proactively manage this:

- Automated API health checks: Daily test calls to each platform API to detect breaking changes early
- Changelog monitoring: Subscribe to Meta, LinkedIn, and Google developer blogs and API changelog feeds
- Version pinning: All API calls use explicit API version parameters (e.g., Graph API v19.0) to avoid surprise changes
- Deprecation alerts: When a platform announces deprecation, we schedule connector updates within the deprecation window



20. Phase 2 Roadmap

The following features are explicitly out of scope for Phase 1 but architecturally planned for:

Feature	Priority	Estimated Effort	Dependencies
Meta Ads integration	High	2–3 weeks	Ads API approval, additional OAuth scopes
Google Ads integration	High	2–3 weeks	Google Ads API access, conversion tracking schema
LinkedIn Ads integration	Medium	2 weeks	LinkedIn Marketing Partner approval
Custom report builder	High	3–4 weeks	Drag-and-drop report designer, PDF export
YouTube Analytics	Medium	2 weeks	YouTube Data API, additional connector
Google Search Console	Medium	1–2 weeks	GSC API, additional connector
CRM integration (HubSpot, Salesforce)	Medium	3–4 weeks	Lead attribution, pipeline mapping
White-label / custom branding	Medium	2–3 weeks	Per-tenant theming, custom domains (CNAME)
Advanced anomaly detection	Medium	3–4 weeks	Statistical models, automated alert rules
AI recommendations	High	4–6 weeks	Actionable recommendations beyond Q&A (e.g., "Post more on Tuesdays")
Mobile app (React Native)	Low	8–12 weeks	Shared API, push notifications
Webhook notifications	Low	1 week	Event system, webhook management UI
Data export API	Medium	1–2 weeks	CSV/JSON export, scheduled reports
SSO (SAML/OIDC)	Medium	2–3 weeks	Enterprise auth, identity provider integration

Extensibility Architecture

The system is designed for easy platform additions:

```
// Adding a new platform (e.g., YouTube) requires:  
// 1. Create YouTubeConnector extending BaseConnector (1 file)  
// 2. Register in connector factory (1 line)  
// 3. Add platform enum value (1 line)  
// 4. Database migration adds 'youtube' to platform check constraint (1 migration)  
// 5. Frontend adds YouTube icon/color to platform components (2-3 files)  
  
// Total effort for a new platform connector: ~1-2 weeks
```



21. Assumptions & Clarification Points

Assumptions Made

#	Assumption	Impact if Wrong
A1	Client has or will create developer apps on Meta, LinkedIn, and Google Cloud before Phase 2 starts	Phase 2 blocked; we can assist with app registration
A2	Instagram data is accessed via Meta Graph API (Business/Creator accounts, not personal)	Different API required for personal accounts
A3	Initial historical data pull covers 90 days (not years of history)	Longer history = higher API quota usage and longer initial sync
A4	English-only UI for Phase 1	i18n adds 1–2 weeks if needed
A5	Single region deployment (us-east-1) initially	Multi-region adds infrastructure complexity and cost
A6	No SSO requirement in Phase 1 (email/password auth)	SSO adds 2–3 weeks
A7	Platform API sandbox/test accounts available for development	Development uses real accounts if sandbox unavailable
A8	Maximum 50 connected accounts per organization in Phase 1	Higher limits require queue partitioning adjustments
A9	Budget allows for Datadog-tier observability (not just CloudWatch)	CloudWatch alternative reduces cost but limits visibility
A10	Client team has AWS account access for infrastructure provisioning	We can use our AWS account with cross-account access

Clarification Points Needed

#	Question	Impact on Design
C1	How much historical data should the initial sync pull? (30 days? 90 days? All available?)	Affects initial sync duration, API quota planning, and storage estimates
C2	Are there specific branding/theming requirements for the dashboard?	Affects UI development scope
C3	Should agencies be able to have their own billing, or is this managed centrally?	Affects organization hierarchy design
C4	What is the expected tenant count at launch and 6-month mark?	Affects initial infrastructure sizing
C5	Are there any compliance requirements (SOC 2, HIPAA, ISO 27001)?	Affects security controls and audit scope
C6	Does the client have an existing AWS account, or should we provision new?	Affects IaC setup and billing
C7	Is there a preference for the frontend hosting (Vercel vs. self-hosted on AWS)?	Affects deployment pipeline design
C8	Should the AI insights layer support multiple languages or English only?	Affects prompt engineering scope
C9	What is the expected data volume per tenant? (Number of posts/day, number of connected accounts)	Affects database sizing and partition strategy
C10	Is there an existing design system or brand guidelines to follow?	Affects UI development approach



22. Answers to Proposal Questions

Q1. Similar work

Bilzee is the closest relevant example from our SaaS work: a multi-tenant billing/CRM platform with organization/store-level separation, role-based access, configurable modules, reporting, invoicing and cloud deployment. The difficult parts were tenant-safe data access, configurable business rules, reliable document generation, deployment reliability and keeping the platform extensible as requirements grew.

Our logistics product work is also relevant from a data-platform perspective: high-volume operational workflows, API-driven data, complex filtering/pagination, time-slot logic and reliability concerns. It is not presented as a direct marketing-analytics equivalent. We prefer to be explicit about that distinction rather than overstate similarity.

For this brief, the genuinely hard parts are provider API differences, historical consistency, rate limits/token lifecycle, normalization across incompatible metric definitions, idempotent syncs, and maintaining strict tenant isolation through both synchronous APIs and background workers.

Q2. Tech stack

Frontend: Angular. Backend: NestJS/Node.js. Database: PostgreSQL. Async processing: Redis-backed queue/workers. Cloud: Azure. Secrets: Azure Key Vault. Observability: Azure Monitor/Application Insights.

This stack aligns well with our existing engineering strengths in Angular, Node.js/NestJS, PostgreSQL, Azure, SaaS architecture, API-driven applications and production CI/CD. PostgreSQL is particularly appropriate because the platform is fundamentally relational and reporting-heavy, while also supporting JSON fields for provider-specific metadata.

Q3. Multi-tenancy approach

We recommend shared PostgreSQL infrastructure with explicit organization-level tenant keys, strict service/repository scoping and PostgreSQL Row-Level Security as defense in depth for critical tables. The browser never supplies a trusted tenant boundary; the server derives it from authenticated organization membership.

Background jobs are equally tenant-scoped. Every job carries organization and connection context, and worker code validates ownership before reading or writing data. Automated negative tests attempt cross-tenant access as part of the release pipeline.

For very large enterprise tenants, the architecture can later support partitioning or dedicated database/schema isolation without changing the product's external tenant model.

Q4. Team & structure

Recommended dedicated delivery pod:

- 1 Solution/Technical Architect – architecture, security, data model and technical governance.
- 1 Senior Backend Engineer – NestJS, PostgreSQL, integrations and data pipeline.
- 1 Backend/Integration Engineer – provider connectors, OAuth and sync workers.
- 1 Senior Frontend Engineer – Angular dashboard and UX implementation.
- 1 QA/Automation Engineer – API, integration, tenant-isolation, regression and performance testing.
- 1 UI/UX resource part-time during discovery/dashboard phases.
- 1 Project/Delivery Manager part-time for planning, communication, risk and release coordination.

The core engineering pod should be dedicated during active implementation rather than split across unrelated delivery commitments. Specialist support can be shared where appropriate.

Q5. Timeline

The realistic core delivery window is approximately 16–21 calendar weeks, with the phases listed in Section 18. We would not recommend compressing the project by removing the reliability/data-quality phase because the stored historical layer is central to the product's value and the AI layer depends on trustworthy normalized data.

Q6. Cost

Our planning estimate is ₹40–65 L for the complete scope described, including the core dashboard and initial natural-language insights layer. A Phase 1 dashboard-only implementation excluding AI is approximately ₹28–36 L. Final fixed pricing should follow a short discovery phase and confirmation of provider access, exact reporting dimensions, UX, environments and acceptance criteria.

Recommended billing: milestone-based fixed price for agreed scope, with change requests priced separately. A time-and-materials model is appropriate for Phase 2 expansion and ongoing product evolution.

Q7. Maintenance

Post-launch support should cover application defects, infrastructure monitoring, security updates, provider API changes, connector failures and operational support. Third-party cloud/API charges remain separate.

A new metric/field is assessed for connector, normalization, database, API and UI impact. A new platform is treated as a separate integration work package because OAuth, API approval, rate limits, data mapping, backfill and QA can materially vary by provider.

Q8. AI-assisted delivery

Our team uses AI-assisted development as an engineering productivity layer for code generation, refactoring, test generation, documentation, debugging assistance and implementation acceleration. We still retain human review, architecture ownership, security review and QA as mandatory controls.

For a project like this, AI can reduce repetitive implementation effort, but it should not be used to shortcut provider integration validation, tenant-isolation review or data-quality testing. The expected benefit is faster delivery and smaller amounts of repetitive engineering work, not elimination of senior engineering ownership.

Q9. Backend platform experience

Our preference for this brief is a fully controlled backend rather than making a generated-backend platform the core system of record. Platforms such as Supabase/Firebase/Xano can accelerate CRUD and authentication, but this product's difficult areas are connector orchestration, provider-specific sync behavior, historical normalization, retries, tenant isolation and analytical queries.

A managed PostgreSQL foundation can still be used where it improves speed, but the domain services, connector layer and worker architecture should remain under application-level control. That gives the project a stronger balance between delivery speed and long-term control.

Q10. Infra & hosting

We recommend Azure because it fits our existing operational experience and supports managed application hosting, PostgreSQL, Redis/cache, Key Vault, monitoring and scalable worker services.

At moderate scale, infrastructure is better estimated as a shared platform cost rather than a literal isolated server per tenant. As a planning range, a small-to-moderate tenant may contribute roughly ₹1,500–₹4,000/month of incremental shared infrastructure cost depending on sync frequency, historical retention, API volume and workload. This excludes AI token costs and major enterprise requirements. Actual cloud cost should be validated with a capacity model after expected tenant count and data volume are confirmed.

Q11. AI/agent build

We can build the natural-language layer as an application-controlled analytics assistant rather than giving an LLM direct unrestricted database access. The recommended flow is: user question → intent/metric interpretation → semantic metric validation → tenant-scoped query plan → controlled query execution → result validation → grounded response.

The semantic layer should define canonical metrics, dimensions, date semantics and supported relationships. The LLM is responsible for language interpretation; deterministic services remain responsible for authorization, query constraints and calculation integrity.

A practical initial AI work package is 4–6 weeks with a small specialist team: one AI/backend engineer, one senior backend/data engineer and part-time architecture/QA support. MCP or similar tool protocols can be evaluated where they provide clean tool boundaries, but MCP is not a prerequisite for a secure first release.

Relative to the overall build, AI is approximately 15–25% of the implementation effort if limited to grounded Q&A and explanations. More advanced autonomous recommendations, campaign actions or agentic workflows would be a separate scope.

Q12. Risks

- Provider API limitations and changes — platform APIs may expose different metrics, retention windows and permissions.
- Historical backfill — availability and granularity of historical data can vary by provider/account.
- Metric semantics — identical words such as engagement, reach or impressions may not be directly comparable across platforms.
- OAuth/re-authentication — tokens can expire, permissions can change and accounts can disconnect.
- Rate limits — connector design must support provider-specific throttling and retries.
- Data quality — changes in provider schemas can create silent reporting discrepancies without validation.
- AI grounding — natural-language answers are only as reliable as the canonical metric model and stored data.
- Scale assumptions — tenant count, connected accounts, sync frequency and retention were not specified in the brief.
- Reporting scope — the exact dashboard KPIs, charts, exports and comparison formulas need final acceptance criteria.
- Third-party approvals — production integration access may require provider review or application approval.

Get in Touch



Company

Alphabetum Private Limited

www.alphabetumtech.com

[LinkedIn: Alphabetum Technology](#)



CTO — Shivani Gupta

[LinkedIn: shivaaniguptaa](#)



shivani@alphabetumtech.com



[+91 98975 77556](tel:+919897577556)

